PACMAN ASSIGNMENT

Question 7 (10 marks): Consistency Implies Admissibility

Let h(n) be a heuristic for node n. A heuristic is admissible if $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost from n to the goal. If a heuristic is consistent, for every pair of nodes (ni, ni+1), we have:

- $h(ni) \leq c(ni, ni+1) + h(ni+1)$

Since h(goal) = 0, summing over all nodes gives $h(n0) \leq \Sigma\, c(ni, ni+1) = h^*(n)$. Thus, every consistent heuristic is admissible.

- Example of admissible but inconsistent heuristic:
- h(S) = 2, h(A) = 0

Both are admissible since they do not overestimate actual costs, but S → A violates consistency because $2 \leq 1 + 0$ is false.

Question 9 (5 marks): Inadmissible Heuristic in A*

Consider the following search tree:
S → A (cost 1)
S → B (cost 2)
A → G1 (cost 1)
B → G2 (cost 1)

True path costs: S-A-G1 = 2, S-B-G2 = 3.
Heuristic values: h(S)=4, h(A)=0, h(B)=0.

Here, f(S)=4, f(A)=1, and f(B)=2. A* expands A first and finds S-A-G1 (cost 2). Because the heuristic at S overestimates, A* terminates prematurely, returning a suboptimal path.

Question 10 (10 marks): Negative Edge Cost Paradox

State space: S, A, B, G
Edges:
S→A: 1, A→G: 1, S→B: 1, B→G: 1, A→S: -3, B→S: -3.

The loop S→A→S yields a cost of -2. Repeating this loop infinitely results in a total cost approaching negative infinity. Hence, even with a finite state space, the optimal path never reaches the goal in finite time.

Question 11 (5 marks): Symbol Search Problem

State space: All symbol sequences (length ≤ 5) from an alphabet of over 1000 symbols.
Initial state: Empty sequence.

Goal: Any valid sequence that satisfies constraints.
Cost: Uniform (1 per symbol).

Choice of Algorithm: Breadth-First Search (BFS) is preferred because it efficiently explores shallower levels and avoids getting stuck in deep branches.

Improvement: Use Iterative Deepening Search (IDS) to guarantee completeness, optimality, and memory efficiency.

Question 12 (5 marks): Lowest-Cost Iterative Deepening

To adapt IDS for cost optimization, convert it into a Cost-Limited Search, exploring all paths with total cost ≤ bound C.

Algorithm properties:
• Gradually increase C until the optimal cost C* is found.
• With non-negative costs, this guarantees discovery of the lowest-cost solution.

Required information: cost function, non-negative edge costs. This ensures the first found solution is optimal.